

Absolutely. If your goal is **not just to build an application, but to use that application as a playground for building a production-style Playwright/TypeScript automation framework**, then we should design the application specifically to expose the kinds of problems a robust framework needs to handle.

I'd recommend a **moderate-complexity business application** rather than a simple demo app.

1. Application Concept

Healthcare Operations Management System

A web application where different users can manage patients, appointments, prescriptions, pharmacies, orders, and reports.

It gives us enough complexity to test:

- Authentication & authorization
 - CRUD
 - Forms & validations
 - Dynamic UI
 - Tables
 - Pagination
 - Search/filter/sort
 - File upload/download
 - API interactions
 - Database persistence
 - Role-based access
 - Notifications
 - Modals
 - Iframes
 - Multiple tabs/windows
 - Date/time handling
 - Async operations
 - Error handling
 - Loading states
 - Session handling
 - Audit logs
 - Reports
 - Complex workflows
-

2. Application Modules

A. Authentication

1. Login
 2. Logout
 3. Registration
 4. Forgot password
 5. Reset password
 6. Change password
 7. Remember me
 8. Session timeout
 9. Invalid credentials
 10. Account lock after multiple failures
 11. Role-based login
 12. Access token/session handling
-

B. Dashboard

After login:

- Total patients
- Today's appointments
- Pending prescriptions
- Pending orders
- Completed orders
- Low-stock medicines
- Recent activities
- Notifications
- Quick actions
- Charts
- Date-range filter

Example:

```
-----
| Patients | Appointments | Orders | Prescriptions |
-----
|           Dashboard Charts           |
-----
| Recent Activities           | Notifications           |
-----
```

3. Patient Management

Patient List

- Search
- Filter
- Sort
- Pagination
- Column selection
- Status filter
- Date filter

Patient CRUD

- Create patient
- View patient
- Edit patient
- Delete/deactivate patient

Patient Details

- Personal information
 - Contact information
 - Address
 - Emergency contact
 - Medical-related profile information
 - Dependents
 - Documents
 - Appointment history
 - Prescription history
 - Order history
-

4. Appointment Management

This will be particularly useful for automation.

Features:

- Book appointment
- Reschedule
- Cancel
- Confirm
- Complete

- No-show
- Search appointments
- Filter by date
- Filter by doctor
- Filter by status

Appointment workflow

```
graph TD
    A[Available] --> B[Booked]
    B --> C[Confirmed]
    C --> D[In Progress]
    D --> E[Completed]
```

Alternative:

```
graph LR
    A[Booked] --> B[Cancelled]
    A --> C[No Show]
```

This gives us excellent **state-transition testing**.

5. Doctor / Provider Management

- Provider list
- Add provider
- Edit provider
- Activate/deactivate provider
- Specialization
- Availability
- Working hours
- Appointment slots

Example:

```
classDiagram
    class Doctor {
        Name
        Specialization
        Location
        Available Days
        Start Time
    }
```

6. Pharmacy Management

- Add pharmacy
- Edit pharmacy
- Activate/deactivate
- Pharmacy details
- Address
- Contact
- Operating hours

Also:

Pharmacy-specific URL

For example:

/pharmacy/123

This can help us test:

- URL handling
- Authorization
- Direct navigation
- ID-based routes

7. Prescription Management

Features:

- Create prescription
- Add medicines
- Dosage
- Frequency
- Duration
- Instructions
- Upload prescription document
- View prescription
- Download prescription
- Approve/reject prescription

Example:

```
Prescription
|
+-- Medicine A
|   |--- Dosage
|   |--- Frequency
|   |--- Duration
|
+-- Medicine B
    |--- Dosage
    |--- Frequency
    |--- Duration
```

This gives us **nested forms** and dynamic fields.

8. Medicine / Product Management

This is useful for table automation.

Features:

- Product list
- Add product
- Edit product
- Delete/deactivate
- Search
- Sort
- Filter
- Pagination
- Category
- Manufacturer
- Price
- Stock
- Expiry date

Inventory

- Stock available
 - Low stock
 - Out of stock
 - Stock adjustment
 - Stock history
-

9. Order Management

This should be one of our major workflows.

```
graph TD; A[Prescription] --> B[Create Order]; B --> C[Pharmacy Processing]; C --> D[Packed]; D --> E[Shipped]; E --> F[Delivered];
```

Prescription
↓
Create Order
↓
Pharmacy Processing
↓
Packed
↓
Shipped
↓
Delivered

Possible states:

Pending
Processing
Packed
Shipped
Delivered
Cancelled
Failed

This is excellent for **end-to-end workflow automation**.

10. Reports

Reports page containing:

- Patient report
- Appointment report
- Order report
- Prescription report
- Inventory report

Features:

- Date range
- Filters
- Generate report
- Export CSV
- Export PDF
- Download report

This gives us:

- File downloads
 - File validation
 - Dynamic tables
 - API validation
-

11. Notifications

Support:

- Success notification
- Error notification
- Warning notification
- Info notification

Examples:

Patient created successfully
Order cancelled successfully
Stock level is low
Invalid prescription

Notifications can disappear automatically.

Perfect for testing:

- Toast messages
 - Timing
 - Auto-dismiss
 - Dynamic locators
-

12. Documents

A dedicated document functionality.

Upload

Support:

- PDF
- JPG
- PNG
- CSV

Download

- Prescription
- Patient document
- Report
- Invoice

Negative scenarios

- Unsupported file
- File too large
- Empty file
- Invalid filename

Great for Playwright's:

```
setInputFiles()  
waitForEvent("download")
```

13. Dynamic UI

We should intentionally include:

AJAX/API loading

```
Loading...  
  ↓  
API response  
  ↓  
Data displayed
```

Delayed elements

Some elements appear after:

- 1 sec
- 2 sec
- API response

This helps us test **proper synchronization instead of `waitForTimeout()`**.

14. Modal Dialogs

Examples:

Delete confirmation

Are you sure you want to delete this patient?

[Cancel] [Delete]

Edit modal

Add product modal

Confirmation modal

Warning modal

This gives us good modal-handling scenarios.

15. Tables

We should deliberately create advanced tables.

Features:

- Sorting
- Pagination
- Search
- Filtering
- Dynamic rows
- Row actions
- Checkbox selection
- Select all
- Column visibility
- Empty state

Example:

Patient	Email	Status	Created	Actions
John	...	Active	...	Edit Delete
Mike	...	Active	...	Edit Delete

This becomes excellent material for **generic Table utility/component** in our framework.

16. Iframes

At least one real business use case should contain an iframe.

For example:

```
Payment
  ↓
Third-party payment iframe
```

Inside iframe:

- Card number
- Expiry
- CVV
- Submit

This allows us to practice:

```
TypeScript
frameLocator()
```

17. Multiple Tabs / Windows

Example:

```
Order
  ↓
Track Shipment
  ↓
External Tracking Page
```

Clicking "Track Shipment" opens another tab.

We can test:

```
TypeScript
context.waitForEvent("page")
```

18. Browser Dialogs

Include scenarios for:

- Alert
- Confirm
- Prompt

Example:

```
Are you sure you want to cancel this order?
```

Useful for:

TypeScript

```
page.on("dialog")
```

19. API Layer

This is **very important** if we're building a serious framework.

Backend should expose APIs such as:

```
POST    /api/auth/login
POST    /api/auth/register

GET      /api/patients
POST     /api/patients
GET      /api/patients/:id
PUT      /api/patients/:id
DELETE   /api/patients/:id

GET      /api/appointments
POST     /api/appointments

GET      /api/orders
POST     /api/orders
PUT      /api/orders/:id/status
```

Then our Playwright framework can perform:

UI + API hybrid testing

For example:

```
API → Create patient
      ↓
UI → Verify patient
```

↓

That's much closer to an enterprise automation framework.

20. Database

We should have a real database.

Something like:

```
Users
Patients
Doctors
Appointments
Pharmacies
Products
Inventory
Prescriptions
Orders
Documents
Notifications
AuditLogs
```

This allows us to validate:

```
UI
↓
API
↓
Database
```

21. Role-Based Access Control

At least these roles:

Admin

Everything.

Doctor

Patients + appointments + prescriptions.

Pharmacist

Prescriptions + orders + inventory.

Support

Patients + appointments + limited information.

Viewer

Read-only access.

Then we can test:

```
Admin → Can create/delete
Doctor → Can create prescription
Pharmacist → Can process order
Viewer → Cannot modify
```

This will be **extremely valuable for framework design**.

22. Audit Logs

Every important operation should generate an audit entry.

Example:

```
User: Vivek
Action: UPDATE
Entity: Patient
Entity ID: P1024
Timestamp: ...
```

This gives us another API/database validation opportunity.

23. Error Handling

We should deliberately build:

400

Bad request

401

Unauthorized

403

Forbidden

404

Not found

409

Conflict

500

Internal server error

UI should display meaningful error messages.

This will allow us to build robust negative test scenarios.

24. Search & Filtering

Search should support:

- Exact search
- Partial search
- Case-insensitive search
- No-result search
- Special characters
- Multiple filters
- Clear filters

Example:

```
Search: John
Status: Active
Date: Last 30 days
```

25. Validation

We should have both:

Client-side validation

```
Required field
Invalid email
Invalid phone
```

Server-side validation

Even if UI validation is bypassed:

```
API → 400
```

This is important because a robust automation framework should not assume the UI is the only validation layer.

26. Test Data Scenarios

Application should support:

- Unique data
- Duplicate data
- Boundary values
- Invalid data
- Empty data
- Large data
- Special characters

Example:

```
Valid email
Duplicate email
Invalid email
Empty email
Very long email
```

This will allow us to build a proper **Test Data Factory**.

27. Environment Support

Application should be deployable as:

```
QA
STAGE
PRE-PROD
```

Our Playwright framework can then do:

Bash

```
npm run test:qa  
npm run test:stage  
npm run test:preprod
```

Which fits perfectly with the framework work we've already been doing.

28. Application-Level Scenarios

We should have realistic end-to-end workflows.

Workflow 1 — Patient

```
Register  
↓  
Login  
↓  
Create Patient  
↓  
Edit Patient  
↓  
Verify Patient  
↓  
Deactivate Patient
```

Workflow 2 — Appointment

```
Login  
↓  
Select Patient  
↓  
Select Doctor  
↓  
Select Date  
↓  
Select Slot  
↓  
Book Appointment  
↓  
Confirm
```

Workflow 3 — Prescription

```
Doctor Login  
↓
```

```
Open Patient
↓
Create Prescription
↓
Add Medicines
↓
Upload Document
↓
Submit
```

Workflow 4 — Order

```
Pharmacist Login
↓
Open Prescription
↓
Create Order
↓
Process
↓
Pack
↓
Ship
↓
Deliver
```

29. Automation Framework Capabilities We Can Build Against It

This is the **real objective**.

Our Playwright framework should eventually contain:

```
Framework
|
├─ Config Management
├─ Environment Management
├─ Browser Management
├─ Fixtures
├─ Page Objects
├─ Components
├─ API Clients
├─ Test Data Factory
├─ Database Utilities
├─ Authentication
├─ Storage State
├─ Wait Utilities
```

- └─ Locator Strategy
- └─ Logging
- └─ Screenshots
- └─ Videos
- └─ Traces
- └─ Retry
- └─ Parallel Execution
- └─ Worker Management
- └─ Test Tags
- └─ Test Hooks
- └─ Reporting
- └─ Allure
- └─ CI/CD
- └─ Secrets Management
- └─ Error Handling
- └─ Configuration Validation

30. Advanced Framework Concepts

Once the basic framework works, we can introduce:

Fixture hierarchy

```
Playwright Test
  ↓
Custom Fixtures
  ↓
Auth Fixture
  ↓
Page Fixtures
  ↓
API Fixture
  ↓
Database Fixture
```

Worker-level resources

For expensive resources.

Test-level resources

For isolated test data.

Authentication states

```
admin.json
doctor.json
```

pharmacist.json

API + UI combination

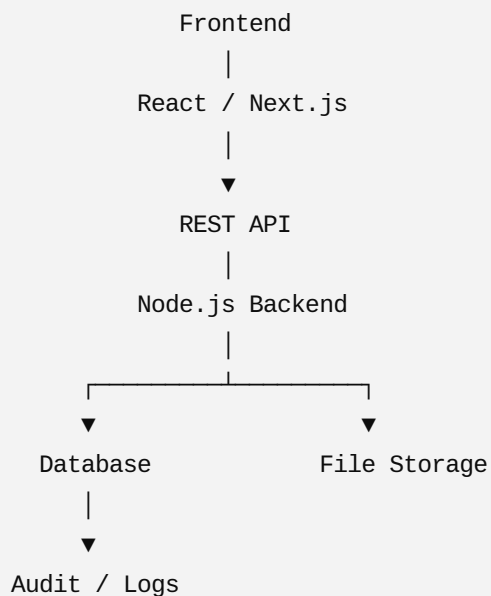
```
API setup
  ↓
UI validation
  ↓
API cleanup
```

Parallel execution

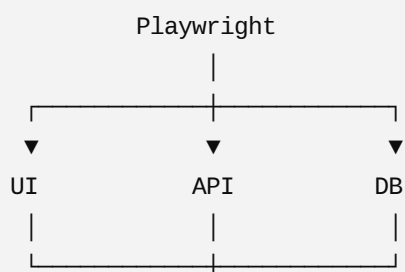
```
Worker 1 → Test A
Worker 2 → Test B
Worker 3 → Test C
Worker 4 → Test D
```

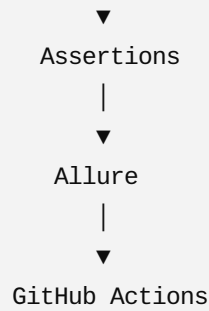
31. Application Architecture

I'd suggest:



For our automation framework:





My recommendation

Don't make the application **too huge initially**.

Let's build it in **phases**:

Phase	What we build
1	Authentication + Dashboard
2	Patient + Doctor management
3	Appointment management
4	Prescription + Pharmacy
5	Products + Inventory
6	Orders + workflow
7	Reports + documents
8	API + Database
9	Roles + permissions
10	Error scenarios + edge cases
11	Advanced UI components
12	Build the enterprise-grade Playwright framework against it

The key is that the application itself should intentionally contain both normal and difficult scenarios. That way, when we build the framework, every framework feature has a real reason to exist rather than being added just because "enterprise frameworks usually have it."

